

Here is the comprehensive guide, formatted for easy copying and pasting into Google Docs.

Weekend Data Engineering Project: Security Log ELT Pipeline

1. Project Summary & Learning Outcomes

This project simulates a simplified security log data pipeline on your local MacBook Pro. You will ingest raw security logs, perform an ETL process to a transactional database (OLTP), and then an ELT process to an analytical data warehouse (OLAP), demonstrating core data engineering concepts.

Purpose:

- **Practical Application:** Get hands-on experience building a end-to-end data pipeline.
- **Core Concepts:** Understand and implement ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) methodologies.
- **Database Architectures:** Differentiate and apply OLTP (Online Transactional Processing) and OLAP (Online Analytical Processing) database concepts.
- **Modern Python Tooling:** Utilize FastAPI for API development, uv for dependency management, and ruff for code linting.
- **Containerization:** Learn basic Docker for deploying services.
- **Data Modeling:** Implement a basic star schema for analytical queries.

What You Will Learn:

- How to design and implement simple data ingestion APIs.
- How to manage Python project dependencies efficiently with uv.
- Best practices for code quality using ruff.
- The difference between transactional (OLTP) and analytical (OLAP) database usage.
- Techniques for transforming raw data into a structured dimensional model.
- Basic containerization with Docker.
- Setting up a local development environment for data engineering tasks.

2. Laptop Preparation Checklist

Ensure your MacBook Pro is set up with the following free tools:

- **Python 3.13:**
 - Download from: <https://www.python.org/downloads/>
 - Verify installation: Open Terminal and run `python3.13 --version`.
- **uv (Ultrafast Python Package Installer):**
 - Install via curl: `curl -LsSf https://astral.sh/uv/install.sh | sh`

- Ensure uv is in your PATH. You might need to add export PATH="\$HOME/.cargo/bin:\$PATH" to your ~/.zshrc or ~/.bash_profile.
 - Verify installation: uv --version
 - **Docker Desktop for Mac:**
 - Download from: <https://www.docker.com/products/docker-desktop/>
 - Install and ensure it's running before starting the project.
 - Verify installation: docker --version
 - **Code Editor / IDE:**
 - **VS Code (Visual Studio Code):** Highly recommended. Download from: <https://code.visualstudio.com/>
 - **PyCharm Community Edition:** Another excellent choice. Download from: <https://www.jetbrains.com/pycharm/download/>
 - **Database Browser:**
 - **DB Browser for SQLite:** Essential for viewing your SQLite OLTP database. Download from: <https://sqlitebrowser.org/>
 - **DuckDB CLI (Optional):** If you choose the DuckDB option, you might want to explore the CLI for direct interaction. Download from: <https://duckdb.org/docs/installation/>
-

3. Project Structure

Create a new directory for your project and organize your files as follows:

```
security_log_pipeline/
├── data/
│   ├── logs_oltp.db    # SQLite database for OLTP (raw ingested logs)
│   └── logs_olap.duckdb # DuckDB database for OLAP (transformed dimensional data)
├── src/
│   ├── log_generator.py # Simulates log creation and sending
│   ├── log_ingestor.py  # FastAPI for E & L (to OLTP staging)
│   ├── data_transformer.py # ELT logic: T & L (to OLAP dimensional model)
│   ├── security_analyzer.py # Analysis script from OLAP
│   └── utils.py         # Helper functions (database connections, schema creation)
├── Dockerfile          # For containerizing the log_ingestor
├── requirements.txt     # Python dependencies
├── .ruff.toml          # Ruff linter configuration
└── README.md           # Project description (this document)
```

4. Project Code

Copy the following code into the respective files.

requirements.txt

```
fastapi
uvicorn
requests
pandas
duckdb    # For OLAP database
SQLAlchemy # Optional, but good for ORM/DB Abstraction in larger projects
ruff      # For code linting
```

.ruff.toml

Ini, TOML

```
# Basic Ruff configuration
line-length = 120
select = ["E", "F", "W", "I"] # Error, Flake8, Warning, Isort
ignore = []
target-version = "py313"

[flake8-quotes]
inline-quotes = "single"
```

src/utils.py

Python

```
import sqlite3
import duckdb
import os

# --- OLTP Database (SQLite) ---
```

```
OLTP_DATABASE_PATH = "data/logs_oltp.db"
```

```
def get_oltp_connection():  
    conn = sqlite3.connect(OLTP_DATABASE_PATH)  
    conn.row_factory = sqlite3.Row # Allows accessing columns by name  
    return conn
```

```
def init_oltp_db():  
    conn = get_oltp_connection()  
    cursor = conn.cursor()  
    # Raw logs staging table for ETL flow  
    cursor.execute("""  
        CREATE TABLE IF NOT EXISTS raw_security_logs (  
            id INTEGER PRIMARY KEY AUTOINCREMENT,  
            timestamp TEXT NOT NULL,  
            source_ip TEXT NOT NULL,  
            event_type TEXT NOT NULL,  
            message TEXT,  
            severity TEXT,  
            ingested_at TEXT DEFAULT CURRENT_TIMESTAMP  
        )  
    """)  
    conn.commit()  
    conn.close()  
    print(f"OLTP SQLite database initialized at {OLTP_DATABASE_PATH}")
```

```
# --- OLAP Database (DuckDB) ---
```

```
OLAP_DATABASE_PATH = "data/logs_olap.duckdb"
```

```
def get_olap_connection():  
    # DuckDB connects directly to the file  
    conn = duckdb.connect(database=OLAP_DATABASE_PATH, read_only=False)  
    return conn
```

```
def init_olap_db():  
    conn = get_olap_connection()  
    cursor = conn.cursor()  
  
    # Dimension Table: dim_event_type  
    cursor.execute("""  
        CREATE TABLE IF NOT EXISTS dim_event_type (  
            event_type_key INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        event_name VARCHAR UNIQUE NOT NULL
    )
'''

# Dimension Table: dim_severity
cursor.execute("""
    CREATE TABLE IF NOT EXISTS dim_severity (
        severity_key INTEGER PRIMARY KEY AUTOINCREMENT,
        severity_level VARCHAR UNIQUE NOT NULL
    )
'''

# Dimension Table: dim_ip_address
cursor.execute("""
    CREATE TABLE IF NOT EXISTS dim_ip_address (
        ip_address_key INTEGER PRIMARY KEY AUTOINCREMENT,
        ip_address VARCHAR UNIQUE NOT NULL
    )
'''

# Fact Table: fact_security_events
cursor.execute("""
    CREATE TABLE IF NOT EXISTS fact_security_events (
        event_id BIGINT PRIMARY KEY, -- DuckDB uses BIGINT for auto-increment by default
        timestamp TIMESTAMP NOT NULL,
        event_type_key INTEGER NOT NULL,
        severity_key INTEGER NOT NULL,
        ip_address_key INTEGER NOT NULL,
        message VARCHAR,
        -- FOREIGN KEY constraints are declarative in DuckDB and not enforced for
performance,
        -- but good practice to include for schema clarity.
        FOREIGN KEY (event_type_key) REFERENCES dim_event_type(event_type_key),
        FOREIGN KEY (severity_key) REFERENCES dim_severity(severity_key),
        FOREIGN KEY (ip_address_key) REFERENCES dim_ip_address(ip_address_key)
    )
'''
conn.close() # Important to close DuckDB connections
print(f"OLAP DuckDB database initialized at {OLAP_DATABASE_PATH}")

if __name__ == "__main__":
    # Ensure data directory exists

```

```
os.makedirs("data", exist_ok=True)
init_oltp_db()
init_olap_db()
print("All databases initialized.")
```

src/log_generator.py

Python

```
import time
import random
import datetime
import requests
import json

# This will simulate sending logs to a local HTTP endpoint (your log_ingestor)
INGESTOR_URL = "http://localhost:8000/ingest_log" # FastAPI default port is 8000

def generate_log():
    ip_prefix = "192.168.1."
    event_types = ["LOGIN_ATTEMPT", "FILE_ACCESS", "NETWORK_SCAN", "API_CALL",
"MALWARE_DETECTED"]
    severities = ["INFO", "WARNING", "CRITICAL", "HIGH"]

    event_type = random.choice(event_types)
    source_ip = ip_prefix + str(random.randint(1, 254))
    timestamp = datetime.datetime.now().isoformat()
    severity = random.choice(severities)

    message = f"User '{random.choice(['admin', 'guest', 'user1'])}' from {source_ip} performed a {event_type}."
    if event_type == "MALWARE_DETECTED":
        message = f"Malware detected on host {source_ip}. Threat: {random.choice(['Trojan', 'Ransomware', 'Adware'])}"
        severity = "CRITICAL"
    elif event_type == "NETWORK_SCAN":
        message = f"Unusual network scan detected from {source_ip} targeting internal systems."
        severity = "HIGH"
```

```

log_entry = {
    "timestamp": timestamp,
    "source_ip": source_ip,
    "event_type": event_type,
    "message": message,
    "severity": severity
}
return log_entry

def send_log(log):
    try:
        response = requests.post(INGESTOR_URL, json=log)
        if response.status_code == 200:
            print(f"Sent log: {log['event_type']} from {log['source_ip']}")
        else:
            print(f"Failed to send log: {response.status_code} - {response.text}")
    except requests.exceptions.ConnectionError:
        print("Connection error: Is log_ingestor.py running?")

if __name__ == "__main__":
    print(f"Starting log generator. Sending logs to {INGESTOR_URL}")
    print("Ensure log_ingestor.py is running first.")
    try:
        while True:
            log = generate_log()
            send_log(log)
            time.sleep(random.uniform(0.5, 2.0)) # Send logs every 0.5 to 2 seconds
    except KeyboardInterrupt:
        print("Log generator stopped.")

```

src/log_ingestor.py

Python

```

from fastapi import FastAPI, Request, HTTPException
from pydantic import BaseModel
import sqlite3
import datetime
from .utils import get_oltp_connection, init_oltp_db # Import OLTP specific utilities

app = FastAPI()

```

```

# Pydantic model for incoming log data
class LogEntry(BaseModel):
    timestamp: str
    source_ip: str
    event_type: str
    message: str
    severity: str

# Initialize the OLTP database schema on startup
@app.on_event("startup")
async def startup_event():
    init_oltp_db()
    print("FastAPI app started. OLTP Database schema ensured.")

@app.post("/ingest_log")
async def ingest_log(log_entry: LogEntry):
    conn = get_oltp_connection()
    cursor = conn.cursor()
    try:
        # Basic validation (e.g., timestamp format)
        datetime.datetime.fromisoformat(log_entry.timestamp)

        # Load raw log into the OLTP staging table (raw_security_logs)
        cursor.execute(
            """INSERT INTO raw_security_logs
               (timestamp, source_ip, event_type, message, severity)
               VALUES (?, ?, ?, ?, ?)""",
            (log_entry.timestamp, log_entry.source_ip, log_entry.event_type,
             log_entry.message, log_entry.severity)
        )
        conn.commit()
        return {"status": "Log ingested successfully into OLTP staging"}
    except ValueError:
        raise HTTPException(status_code=400, detail="Invalid timestamp format")
    except sqlite3.Error as e:
        print(f"Database error during ingestion: {e}")
        raise HTTPException(status_code=500, detail="Database error during ingestion")
    finally:
        conn.close()

# To run: uvicorn src.log_ingestor:app --reload --port 8000

```


src/data_transformer.py

Python

```
import sqlite3
import duckdb
import pandas as pd
import random
from .utils import get_oltp_connection, get_olap_connection

def transform_and_load_data():
    # --- 1. Extract (from OLTP) ---
    oltp_conn = get_oltp_connection()
    # Read all raw logs. In a real system, you'd track processed logs.
    raw_logs_df = pd.read_sql_query("SELECT * FROM raw_security_logs ORDER BY
ingested_at ASC", oltp_conn)
    oltp_conn.close()

    if raw_logs_df.empty:
        print("No new raw logs to transform.")
        return

    print(f"\n--- Starting Data Transformation and Loading (ELT 'T' step) ---")
    print(f"Found {len(raw_logs_df)} raw logs to process from OLTP.")

    # --- 2. Load (into OLAP staging - implicitly here, or direct insert) & Transform ---
    olap_conn = get_olap_connection()
    cursor = olap_conn.cursor()

    try:
        # Process each raw log into the dimensional model
        for index, log in raw_logs_df.iterrows():
            # Get or Insert into dim_event_type
            event_type_key = cursor.execute(
                "SELECT event_type_key FROM dim_event_type WHERE event_name = ?",
(log['event_type'],)
            ).fetchone()
            if not event_type_key:
                cursor.execute("INSERT INTO dim_event_type (event_name) VALUES (?)",
(log['event_type'],))
                event_type_key = cursor.execute("SELECT last_insert_rowid()").fetchone()[0]
```

```

else:
    event_type_key = event_type_key[0]

    # Get or Insert into dim_severity
    severity_key = cursor.execute(
        "SELECT severity_key FROM dim_severity WHERE severity_level = ?", (log['severity'],)
    ).fetchone()
    if not severity_key:
        cursor.execute("INSERT INTO dim_severity (severity_level) VALUES (?)",
            (log['severity'],))
        severity_key = cursor.execute("SELECT last_insert_rowid()").fetchone()[0]
    else:
        severity_key = severity_key[0]

    # Get or Insert into dim_ip_address
    ip_address_key = cursor.execute(
        "SELECT ip_address_key FROM dim_ip_address WHERE ip_address = ?",
        (log['source_ip'],)
    ).fetchone()
    if not ip_address_key:
        cursor.execute("INSERT INTO dim_ip_address (ip_address) VALUES (?)",
            (log['source_ip'],))
        ip_address_key = cursor.execute("SELECT last_insert_rowid()").fetchone()[0]
    else:
        ip_address_key = ip_address_key[0]

    # Generate a unique event_id for fact table (can be log['id'] from OLTP if unique
    enough)
    # Or use a simple hash of log content for idempotency, or a timestamp-based ID
    # For simplicity, we'll use the original OLTP ID if possible, otherwise generate a big
    random one
    event_id = log['id'] if 'id' in log and log['id'] is not None else
    random.randint(1000000000000000, 9999999999999999)

    # Insert into fact_security_events
    cursor.execute(
        """INSERT OR IGNORE INTO fact_security_events
            (event_id, timestamp, event_type_key, severity_key, ip_address_key, message)
            VALUES (?, ?, ?, ?, ?, ?)""",
        (event_id, log['timestamp'], event_type_key, severity_key, ip_address_key,
        log['message'])
    )

```

```

    olap_conn.commit()
    print(f"Successfully transformed and loaded {len(raw_logs_df)} raw logs into OLAP
dimensional model.")

```

```

    # --- Clean up OLTP (Optional, but good for ELT) ---
    # After successful transformation, you might clear the raw logs from OLTP
    # olap_conn.close() # Close OLAP connection before modifying OLTP again
    # oltp_conn = get_oltp_connection()
    # oltp_cursor = oltp_conn.cursor()
    # oltp_cursor.execute("DELETE FROM raw_security_logs WHERE id IN (?)",
(tuple(raw_logs_df['id'].tolist()),))
    # oltp_conn.commit()
    # print("Raw logs cleared from OLTP staging table.")

```

```

except Exception as e:
    print(f"Error during transformation: {e}")
    olap_conn.rollback() # Rollback changes if an error occurs
finally:
    olap_conn.close() # Important to close DuckDB connection

```

```

if __name__ == "__main__":
    transform_and_load_data()

```

src/security_analyzer.py

Python

```

import pandas as pd
from .utils import get_olap_connection

def analyze_dimensional_logs():
    olap_conn = get_olap_connection()

    # Join fact with dimension tables for richer analysis
    query = """
SELECT
    f.timestamp,
    dip.ip_address AS source_ip,
    det.event_name AS event_type,
    ds.severity_level AS severity,
    f.message

```

```

FROM
    fact_security_events f
JOIN
    dim_event_type det ON f.event_type_key = det.event_type_key
JOIN
    dim_severity ds ON f.severity_key = ds.severity_key
JOIN
    dim_ip_address dip ON f.ip_address_key = dip.ip_address_key
ORDER BY f.timestamp DESC
,,,,,

# Use DuckDB's pandas integration
df = olap_conn.execute(query).fetchdf()
olap_conn.close()

if df.empty:
    print("No transformed logs to analyze.")
    return

print("\n--- Security Log Analysis (from OLAP DuckDB Dimensional Model) ---")

# 1. Count events by type and severity
print("\nEvent Type Counts:")
print(df['event_type'].value_counts())

print("\nSeverity Counts:")
print(df['severity'].value_counts())

# 2. Identify critical/high severity events
critical_events = df[df['severity'].isin(['CRITICAL', 'HIGH'])]
if not critical_events.empty:
    print("\nCritical/High Severity Events:")
    print(critical_events[['timestamp', 'source_ip', 'event_type', 'message']])
else:
    print("\nNo critical/high severity events detected.")

# 3. Simple detection: Multiple login attempts from same IP (threshold-based)
print("\nPotential Brute-Force Attempts (more than 3 LOGIN_ATTEMPT from same IP):")
login_attempts = df[df['event_type'] == 'LOGIN_ATTEMPT']
if not login_attempts.empty:
    ip_login_counts = login_attempts['source_ip'].value_counts()
    suspicious_ips = ip_login_counts[ip_login_counts > 3] # Threshold of 3
    if not suspicious_ips.empty:
        print(suspicious_ips)

```

```
        print("\nConsider blocking these IPs or investigating further.")
    else:
        print("No suspicious login attempts found.")
    else:
        print("No login attempts logged.")

if __name__ == "__main__":
    analyze_dimensional_logs()
```

Dockerfile

Dockerfile

```
# Use Python 3.13 as the base image
FROM python:3.13-slim-buster

# Set the working directory in the container
WORKDIR /app

# Copy uv installer script
COPY --from=gchr.io/astral-sh/uv:latest /usr/bin/uv /usr/bin/uv

# Copy project files
COPY requirements.txt ./
COPY src/ ./src/

# Create the data directory (for persistent database files)
RUN mkdir -p data

# Install dependencies using uv
RUN uv pip install --no-cache-dir -r requirements.txt

# Expose port 8000 for the FastAPI app
EXPOSE 8000

# Run log_ingestor.py with uvicorn when the container launches
# --host 0.0.0.0 is important to make it accessible from outside the container
CMD ["uvicorn", "src.log_ingestor:app", "--host", "0.0.0.0", "--port", "8000"]
```

5. Guided Instructions to Run

Follow these steps in your MacBook Pro's Terminal:

Step 1: Create Project Directory & Files

Bash

```
mkdir security_log_pipeline
cd security_log_pipeline
# Create all the files from Section 4 (requirements.txt, .ruff.toml, src/*.py, Dockerfile)
# (Alternatively, if you download a zip, extract and navigate into the directory)
```

Step 2: Initialize Git (Optional but Recommended)

Bash

```
git init
```

Step 3: Install Python Dependencies Locally

Bash

```
uv pip install -r requirements.txt
```

Step 4: Initialize Databases

Bash

```
python3.13 -m src.utils
```

- This will create data/logs_oltp.db (SQLite) and data/logs_olap.duckdb (DuckDB) and set up their schemas.
- Use **DB Browser for SQLite** to inspect data/logs_oltp.db. You can use the DuckDB CLI or pandas in a Python script to interact with data/logs_olap.duckdb.

Step 5: Run Code Linting (Ruff)

Bash

uv run ruff check .

- This will check your code for stylistic issues and potential errors based on the .ruff.toml configuration. Fix any issues it reports.

Step 6: Run the Log Ingestor (FastAPI) - Choose ONE method

Method A: Run Locally (for quick testing)

Bash

```
uvicorn src.log_ingestor:app --reload --port 8000
```

- The FastAPI app will start on <http://localhost:8000>. You can open your browser to <http://localhost:8000/docs> to see the interactive API documentation.
- Leave this terminal window open.

Method B: Run with Docker (Recommended for Containerization Experience)

- Ensure **Docker Desktop** is running.
- **Build the Docker image:**

Bash

```
docker build -t log-ingestor-app .
```

- **Run the Docker container:**

Bash

```
docker run -p 8000:8000 -v "$(pwd)/data:/app/data" log-ingestor-app
```

- -p 8000:8000: Maps port 8000 on your host to port 8000 in the container.
- -v "\$(pwd)/data:/app/data": **Crucial for persistence!** This mounts your local data directory into the container. This means both logs_oltp.db and logs_olap.duckdb will persist locally and be accessible by your other scripts.

- Leave this terminal window open.

Step 7: Run the Log Generator (in a NEW Terminal Window)

Bash

```
python3.13 -m src.log_generator
```

- This script will start sending simulated logs to your running log_ingestor (on localhost:8000).
- You should see "Sent log..." messages in this terminal, and incoming log messages in the terminal where your log_ingestor is running.
- Let this run for a few minutes to accumulate some raw data in data/logs_oltp.db.

Step 8: Run the Data Transformer (in a NEW Terminal Window)

Bash

```
python3.13 -m src.data_transformer
```

- Run this *after* some logs have accumulated from the generator.
- This script will extract data from logs_oltp.db, transform it, and load it into logs_olap.duckdb.
- You can run this script multiple times to process new batches of raw logs.

Step 9: Run the Security Analyzer (in a NEW Terminal Window)

Bash

```
python3.13 -m src.security_analyzer
```

- Run this *after* the data_transformer has loaded some data into logs_olap.duckdb.
- This script will connect to your OLAP database and print out various analyses of the transformed security events.

6. Key Concepts Used

- **ETL (Extract, Transform, Load):** Demonstrated by log_ingestor.py. Raw data is extracted from the simulated source (log generator), transformed (minimal validation/structuring), and loaded into the raw_security_logs table (OLTP). This is typically for moving data from a source system to a staging area or directly into an operational database.
- **ELT (Extract, Load, Transform):** Demonstrated by data_transformer.py. Raw data is first extracted from the OLTP raw_security_logs table and *loaded directly* into DuckDB, which acts as a staging area within the OLAP environment. The transformation into the dimensional model then occurs *within* DuckDB using SQL queries. This is common in modern data warehouses where the raw data is loaded first, and transformations happen later.
- **OLTP (Online Transactional Processing):** Represented by logs_oltp.db (SQLite). Optimized for high-volume, concurrent read/write operations and data integrity (e.g., individual log insertions).
- **OLAP (Online Analytical Processing):** Represented by logs_olap.duckdb (DuckDB). Optimized for complex analytical queries, aggregations, and reporting over large datasets. DuckDB is a powerful, in-process, columnar database excellent for local OLAP tasks.

- **Dimensional Modeling (Star Schema):** Used in logs_olap.duckdb. Data is organized into a central "fact" table (fact_security_events) that stores measurements and foreign keys, surrounded by "dimension" tables (dim_event_type, dim_severity, dim_ip_address) that provide descriptive context. This structure makes analytical queries fast and intuitive.
 - **FastAPI:** A modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints. It automatically generates interactive API documentation (Swagger UI/OpenAPI).
 - **uv (Ultrafast Python Package Installer and Resolver):** A new, highly performant tool for managing Python project dependencies, often significantly faster than pip and pip-tools.
 - **ruff (Fast Python Linter):** An extremely fast Python linter and formatter that helps enforce code quality and style best practices.
 - **Docker:** A platform for developing, shipping, and running applications in "containers." Containers package an application and all its dependencies (code, runtime, system tools, libraries) into a single, isolated unit, ensuring consistent behavior across different environments.
-

7. Next Steps to Build On

This project provides a strong foundation. Here are ideas to expand it, including your interest in Kafka and Kubernetes:

Data Pipeline Enhancements

- **Robust Error Handling & Monitoring:** Add more comprehensive try-except blocks and integrate Python's logging module throughout all scripts. Consider tools like Prometheus/Grafana for local monitoring (more complex setup).
- **Idempotency:** Implement mechanisms to ensure that processing the same log multiple times doesn't lead to duplicate data in your OLAP tables (e.g., using unique constraints or "upserts" in DuckDB).
- **Data Quality Checks:** Add more explicit validation rules in log_ingestor.py or data_transformer.py (e.g., validate IP format, ensure severity is from an allowed list).
- **Data Enrichment:** In data_transformer.py, enrich logs by calling a third-party API (e.g., a free IP geolocation API) to add city/country information to the dim_ip_address table.

Scale & Orchestration

- **Apache Kafka (Conceptual):**
 - **Purpose:** For handling high-throughput, real-time streams of events reliably.
 - **Integration:** Instead of directly writing to SQLite, log_ingestor.py would publish log events to a Kafka topic. data_transformer.py (or a new consumer application) would then read from that Kafka topic.

- **Local Setup:** Running Kafka locally is complex (requires Zookeeper). You'd typically use Docker Compose for a multi-container Kafka setup or a managed service like Confluent Cloud for a real project.
- **Orchestration (Airflow/Prefect):** Instead of manually running `data_transformer.py`, use a workflow orchestrator like Apache Airflow or Prefect to schedule and manage your ELT jobs. These can also be run locally with Docker Compose.
- **Data Lake / Data Warehouse:** Instead of just DuckDB locally, research how a cloud data lake (e.g., Amazon S3) and cloud data warehouse (e.g., Snowflake, Google BigQuery, or Amazon Redshift) would fit into this architecture.

Deployment & Operations

- **Kubernetes (Conceptual):**
 - **Purpose:** For orchestrating containerized applications at scale (like your `log_ingestor`).
 - **Integration:** Instead of just `docker run`, you would define Kubernetes YAML manifests for your `log_ingestor` deployment, service, and perhaps persistent volumes for databases (though typically external databases are used with K8s).
 - **Local Setup:** Use Minikube or Docker Desktop's built-in Kubernetes for local clusters. This is a significant learning curve for a weekend.
- **CI/CD Pipeline:** Implement a basic CI/CD pipeline (e.g., using GitHub Actions) to automatically lint, test, build Docker images, and (conceptually) deploy your services.

Advanced Analytics

- **Machine Learning Integration:** Use libraries like `scikit-learn` (or Spark ML if you incorporate Spark later) within `security_analyzer.py` to build simple anomaly detection models (e.g., flagging unusual login patterns).
- **Dashboards:** Build a simple web dashboard using Streamlit or Dash to visualize the analyzed security events from DuckDB.

By tackling these next steps, you can progressively build a more robust and production-ready data engineering project, showcasing a broader range of skills relevant to modern data platforms.
